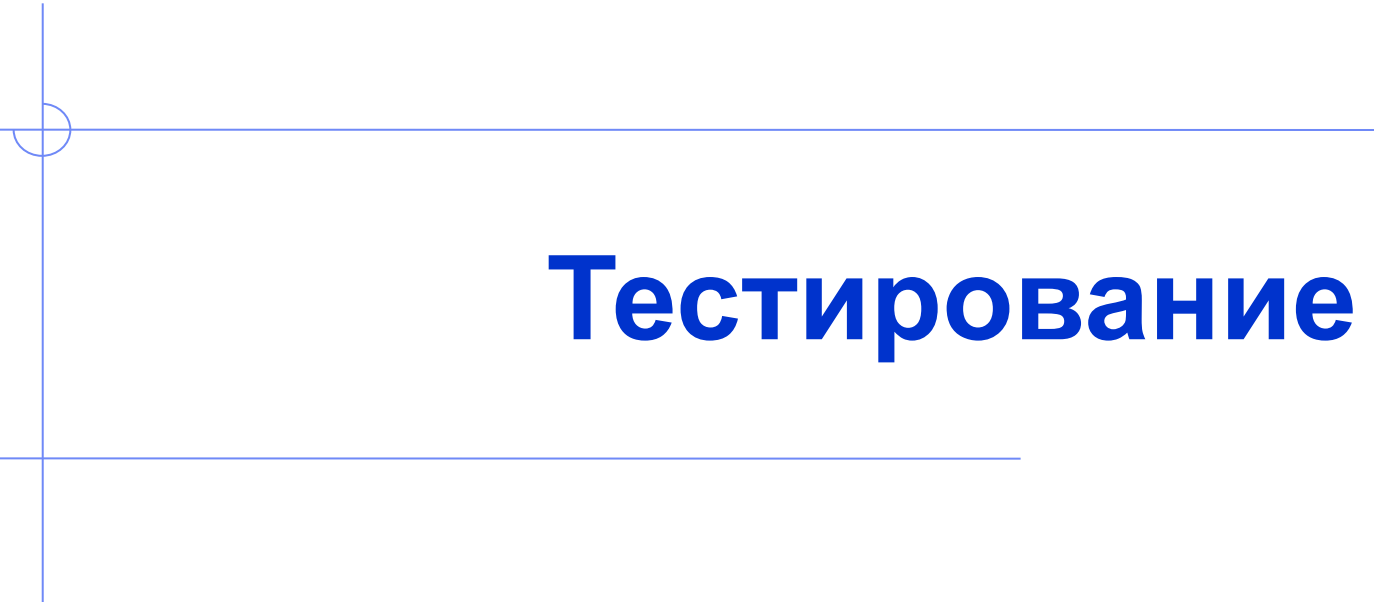




Тестирование



Иванова Татьяна Владимировна
tvivanova@itmo.ru

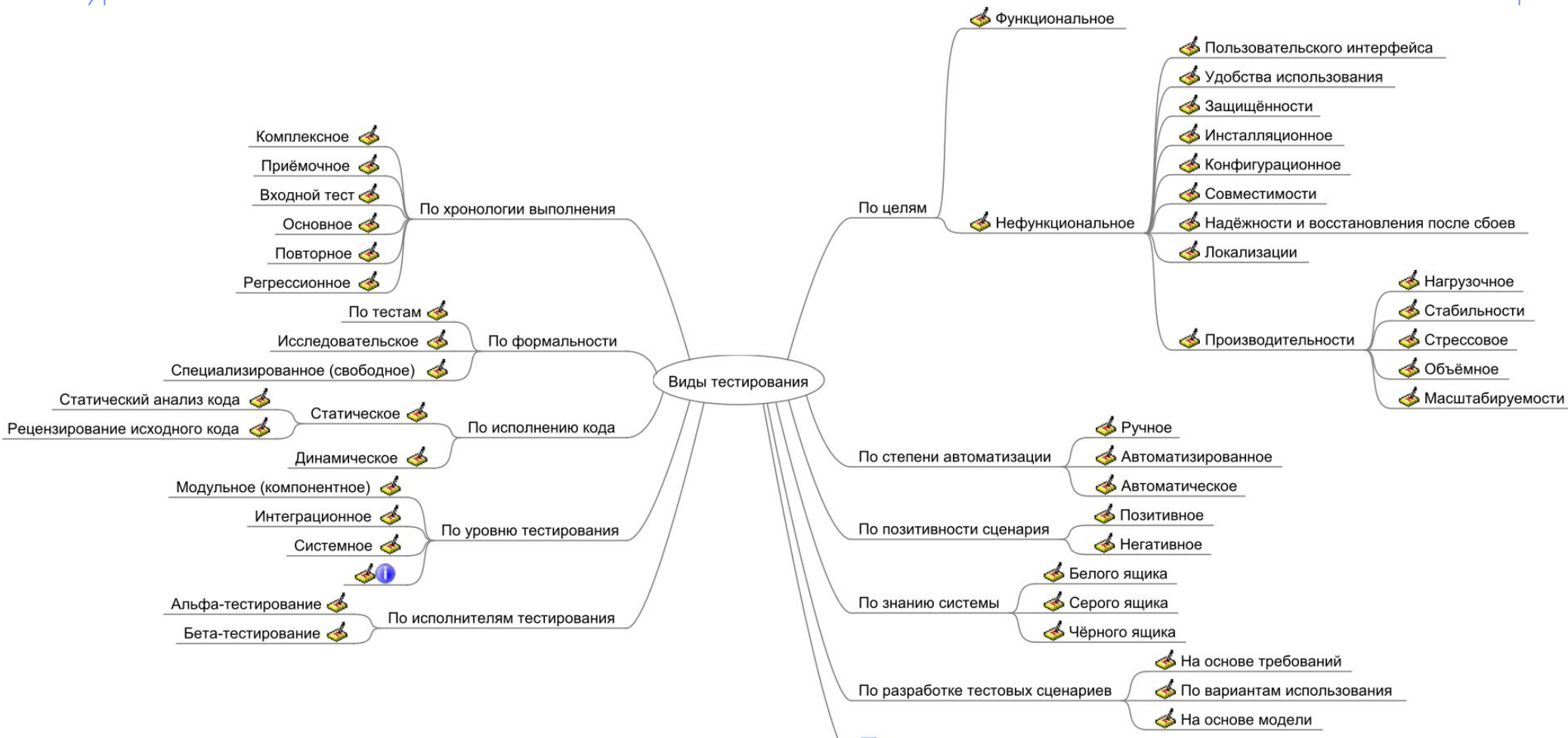
Тестирование

- Тестирование – процесс выявления дефектов (ошибок, багов) в программе
- Дефект - это несоответствие фактического результата выполнения программы **ожидаемому результату (спецификации, User Guid)**
- На этапе тестирования осуществляется и исправление найденных дефектов
 - стадии: разработка – тестирование – релиз

Тестирование

- Кто тестирует
 - разработчики (Developer)
 - тестировщики (Quality Assistant engineer, QA)
 - инженер по применению (?) (Application Engineer)
 - хорошо знает продукт как пользователь, общается с конечными пользователями (проводит консультации, мастер-классы, ...), знает примеры реального использования
 - конечные пользователи (User)
- Зависит от размера компании
- Что тестируем (все!)
 - работу программы
 - быстродействие
 - устойчивость под большой нагрузкой
 - расход ресурсов (оперативная память, место на диске)

Классификация



Основные виды / типы тестирования

- Функциональное тестирование (**Functional Testing**) - проверяется корректность работы функциональности приложения
 - Модульное тестирование (**Unit Testing**) - тестирование программы на уровне отдельно взятых модулей, функций или классов
 - Интеграционное тестирование (**Integration Testing**) - тестирование части системы, состоящей из двух и более модулей
 - Тестирование интерфейса пользователя (**UI testing**) - тестирование элементов управления, проверка, что интерфейс соответствует ожидаемой производительности и функциональности
 - Регрессионное тестирование (**Regression Testing**) - производится при внесении изменений на фазе тестирования или сопровождения продукта (обычно автоматизированное)
 - Приемочное тестирование (**Acceptance Testing**) - тестирование готового продукта конечными пользователями в реальных условиях
- Не функциональное
 - Тестирование производительности (**Performance Testing**) - оценка общей производительности программы, выполняемая при помощи специализированных инструментов (обычно автоматизированное)
 - Тестирование удобства использования (**Usability Testing**) – оценка степени понятности, легкости в изучении и использовании, привлекательности программного продукта для пользователя при условии использования в заданных условиях эксплуатации (ручное)

Модульное тестирование

- Модульное тестирование (Unit Testing)
 - проводится разработчиками
 - тестирование логически выделенного и изолированного модуля (куска) кода (модули программ, классы, функции и т.д.)
 - например: вывод в файл, потом чтение этого же файла, Set/Get, и т.д.
 - Пишем тесты вручную (запускать можно автоматически)
 - пример – л.р.№3
 - Есть специальные платформы
 - <https://learn.microsoft.com/ru-ru/visualstudio/test/writing-unit-tests-for-c-cpp?view=vs-2022>
- Visual Studio включает в себя эти платформы тестирования C++ без дополнительных загрузок:
- Платформа модульного тестирования Майкрософт для C++
 - Google Test
 - Boost.Test
 - CTest

Unit Testing:

тестирование вычислений

- Сравнением с имеющимися аналогами
 - Matlab для математики, специализированное ПО, другие библиотеки (например boost для матриц)
- «Аналитическое» тестирование – создание специальных примеров (**Use Case**) с известными из теории (в том числе из статей) результатами
 - исходная матрица * обратная матрица = единичная матрица
 - умножение матрицы на единичную = самой матрице
 - преобразование Фурье – задаем const, получаем δ -функцию
 - вычисление ФРТ – вначале без aberrаций (сравниваем с Bessinc), потом кому (хорошо определять направление верх-низ, x/y)
 - пример – л.р.№4

Тестирование пользовательского интерфейса

- Тестирование интерфейса пользователя (**UI testing**) – работает ли как ожидается
 - ручное (обычно стажеры)
 - автоматическое
- Тестирование удобства использования (**Usability Testing**) – работает как ожидается, но хотелось бы чтобы работало по-другому
 - ручное
 - субъективное (можно предложить альтернативу)
 - обычно «enhancement»
 - может быть на этапе приемочного тестирования или в процессе эксплуатации
 - нужен сторонний тестировщик (желательно Appl.Eng. или пользователь, можно QA, в самом крайнем случае другой разработчик)
 - пример: установка на производстве
 - пример: my.itmo

Системы отслеживания дефектов

- Система отслеживания дефектов (Bug Tracking System) — позволяет учитывать, контролировать, систематизировать баги и пожелания пользователей, а также следить за процессом устранения этих дефектов и выполнения или невыполнения пожеланий.
 - клиент-серверные (Bugzilla, Jira ...)
 - хостинг (GitHub, Bitbucket, YouTrack ...)
- Обычно баг вносит в систему QA

Состав информации о дефекте

- При создании
 - ID
 - заголовок (короткое описание)
 - тип (Bug / Enhancement / Documentation Defect ...)
 - кто обнаружил
 - дата обнаружения
 - версия продукта
 - критичность дефекта по важности для пользователя, срочности по исправлению
 - Low / Medium / High / Showstopper
 - **подробное** описание как воспроизвести (текст, скрины, файл с примером)
- В процессе работы с дефектом
 - разработчик, который будет исправлять / уже исправил
 - дискуссия (разработчик – QA)
 - текущий статус (New / Attached / In Progress / Fixed / Validation / Closed)
 - версия продукта, в которой исправлено

Жизненный цикл бага

- Разработчик получает сообщение о дефекте, находит ее, исправляет и тестирует
 - возможна дискуссия с QA, доп. вопросы, возврат бага
- Жизненный цикл:
 - New - баг зарегистрирован QA
 - Attached - назначен ответственный за исправление дефекта
 - Fixed - дефект переходит обратно в сферу ответственности QA:
 - Fixed (исправления включены в версию такую-то)
 - Duplicated (повторяет дефект, уже находящийся в работе)
 - Not Bug (работает в соответствии со спецификацией)
 - Unable to Reproduce (запрос дополнительной информации)
 - QA проводит проверку, по результатам:
 - возврат обратно на исправление
 - Validated / Closed – все исправлено
 - Открыт повторно - дефект вновь найден в другой версии

Регрессионное тестирование

- Тестирование после внесения изменений в код приложения (исправление бага, слияние кода, миграция на другую операционную систему, и т.д.)
- Подтверждает, что изменения не внесли новые ошибки (не сломалось в других местах)
 - по типу - Unit Test
 - обычно автоматизированные (запускаются раз в день)
 - Test Case может писать разработчик или QA
 - следят за результатами обычно QA
 - если появилась проблема – создается обычный баг в системе

Для маленьких групп разработчиков

- Если нет QA и Appl.Eng. – что делать?
 - обязательно тщательно тестируем
 - если в группе >1 чел - лучше тестировать друг друга
 - регрессионные тесты можно делать в ручном режиме (?), но регулярно (напр. раз в неделю)
 - если есть возможность - плотно работать с конечным пользователем
 - баги стараться описывать по правилам, но можно в Excel / Google Drive / ... или GitHub и т.д.

Создание итогового ехе и тестирование

Иванова Татьяна Владимировна
tvivanova@itmo.ru

Добавление иконки

- Готовим ico файл

- Проще всего скачать бесплатную иконку 32x32
(например <https://icon-icons.com/ru/pack/Interface-asset/4175>)

- Создаем файл [имя проекта].rc

- указываем полный путь к файлу ico:

```
IDI_ICON1 ICON DISCARDABLE "eye.ico"
```

- Добавляем файл в CMakeLists.txt:

```
set (RESOURCES qtwincalcfourier.ui  
      qtwincalcfourier.rc)
```

- Собираем заново в CMake,
в Visual Studio – Rebuild Solution

- ИТОГО: иконки на диалоговом окне, в проводнике, на рабочем столе, в taskbar

Создание итогового exe приложения

- Берем exe из папки с Release
- Добавляем нужные dll:

Qt6Widgets.dll

Qt6PrintSupport.dll

libfftw3-3.lib

Qt6Core.dll

Qt6Gui.dll

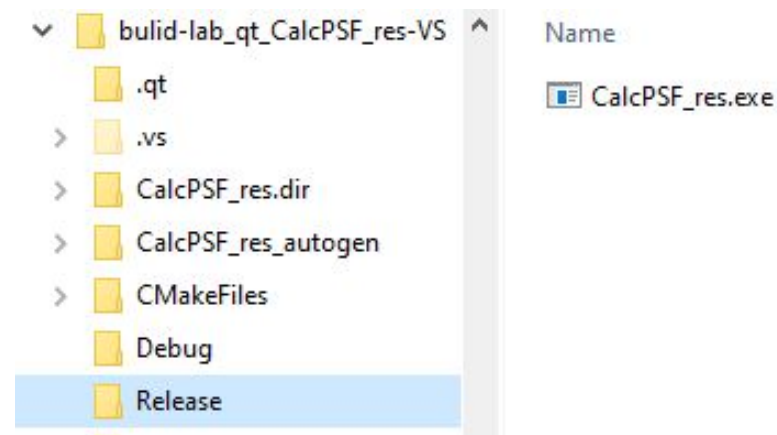
(смотрим что заявлено в CMakeLists.txt:

```
target link libraries(${PROJECT} Qt6::Widgets Qt6::PrintSupport
${FFTW_LIB} ${QT_QTMAIN_LIBRARY})
```

- добавляем еще для Qt в отдельных папках:

platforms/qwindows.dll

styles/qwindowsvistastyle.dll (для красивого отображения)



Лабораторная работа №8

- Создать итоговое приложение

- добавить иконку
- создать zip: exe + dll

(прислать результат через файлообменник или git)

Для тех, кто во-время сдал предыдущие лабы – л.р.№8 не обязательная 😊